

Banco de Dados 2

Álvaro G. Pagliari
alvaro.pagliari@unoesc.edu.br

Roteiro

- Apresentação da disciplina
- Revisão SQL (início)



Apresentação da Disciplina



- **Campus:** 600 – CAMPUS DE CHAPECÓ
- **Curso/Fase:** Sistemas da Informação/12
- **Componente curricular:** 11789 – Banco de Dados II
- **Professor:** 243384 – Álvaro G. Pagliari
- **Nr. créditos/Carga Horária:** 4/72
- **Período letivo:** 2026/2



Apresentação da Disciplina



- Ementa
 - SQL Avançado (uso de indexações, gatilhos, procedimentos, funções, e estruturas de arquivo), gerenciamento de transações e controle de concorrências (mecanismos de proteção e recuperação em caso de falhas). Estudo de caso. Arquitetura do Banco de dados Relacionais (gerenciamentos de transações, armazenamento, Buffer), Otimização de consultas, Bancos distribuídos, Bancos não relacionais (NoSQL, NewSQL) . Atividades práticas em laboratório.



Apresentação da Disciplina



- Justificativa
 - O armazenamento de dados influencia diretamente no desempenho de uma aplicação. Desta forma, é essencial que os profissionais conheçam a arquitetura do banco de dados, e os diferentes paradigmas de banco para que pautado na teoria possa selecionar o modelo de dados mais adequado para cada tipo de problema. A disciplina de Banco de Dados II apresenta aos educandos as ferramentas necessárias para conhecer a fundo a arquitetura de um sistema gerenciador de banco de dados, os diferentes paradigmas de armazenamento e quais suas características .
 - Além disso, deve-se considerar como base a RESOLUÇÃO No 5, DE 16 DE NOVEMBRO DE 2016, que institui as Diretrizes Curriculares Nacionais para os cursos de graduação na área da Computação, a disciplina Banco de Dados I visa contribuir para:
 - compreender, implementar e gerenciar a segurança de sistemas de computação;
 - modelar e implementar soluções de Tecnologia de Informação em variados domínios de aplicação;
 - analisar e avaliar arquiteturas de computadores, incluindo plataformas paralelas e distribuídas, como também desenvolver e otimizar software para elas;



Apresentação da Disciplina



- Objetivo
 - Fornecer ao educando uma visão geral das estruturas internas de um Sistema de Gerência de Banco de Dados (SGBD), bem como apresentar os diferentes paradigmas de armazenamento de dados.



Apresentação da Disciplina

Dia(s) letivo(s)	Conteúdo - Unidade de Ensino	Atividade
28/07/2026	Revisão SQL - Conceitos básicos - Seleção e Joins	No início da aula será apresentado o plano de Ensino. Então iniciaremos uma Revisão de SQL. Aula prática com consultas em uma tabela (projeção e seleção) e consultas em mais de uma tabela (junções). Exercícios práticos.
01/08/2026	Revisão SQL - Funções de Grupo - Consultas Aninhadas	Continuação da Revisão de SQL. Revisaremos agregações (max, min, avg), bem como o funcionamento do group by. Além disso, iremos revisar as consultas aninhadas dando maior ênfase em seu potencial de otimização nas consultas. Serão realizados exercícios práticos,
04/08/2026	SQL Avançado - Functions + Triggers	Explicações sobre funções e suas aplicações em BDs. Funções complexas com triggers, utilização de triggers como ferramenta de auditoria. Exercícios práticos com funções simples e complexas. Aula prática com exercícios.



Apresentação da Disciplina



11/08/2026	SQL Avançado - Índices	Utilização de índices e como planejar e identificar bons índices. Tipos de índices no Postgres. Exercícios Práticos.
18/08/2026	Arquitetura de Banco de Dados - Otimização de Consultas	Estudo do otimizador de consultas do banco de dados. Geração de árvores de consulta e otimização. Otimização na prática do Banco de dados. Uso da ferramenta 'explain'
1 - Autoestudo	Planos de execução e otimização prática de consultas com EXPLAIN/EXPLAIN ANALYZE.	Estudo dirigido sobre a leitura e interpretação de planos de execução no PostgreSQL. Os alunos deverão executar o comando EXPLAIN ANALYZE em pelo menos três consultas de complexidade crescente (uma simples com filtro, uma com junção entre tabelas e uma com agregação/group by), identificando: tipo de varredura utilizada (Seq Scan, Index Scan, Bitmap Scan), tempo estimado versus tempo real de execução, e pontos de gargalo no plano gerado. Como produto final, os alunos devem propor pelo menos uma otimização para uma das consultas analisadas (criação de índice, reescrita da query, etc.) e comparar o plano antes e depois da mudança, documentando a diferença de custo/tempo observada. Atividade não presencial, com entrega via portal de ensino.



Apresentação da Disciplina

01/09/2026	Arquitetura de Dados na Prática: Monólitos vs Microserviços. Banco de dados compartilhado (arquitetura monolítica/MVC clássica) versus "database per service" (arquitetura de microserviços). Desafios de consistência de dados entre serviços distribuídos e introdução ao conceito de polyglot persistence (escolha de diferentes tipos de banco conforme a necessidade de cada serviço).	Discussão dirigida e estudo de caso em grupo: a partir de um sistema fictício, os alunos deverão propor como distribuir os dados entre serviços, qual tipo de banco (relacional, documento, chave-valor) usar em cada parte e quais desafios de consistência e integração surgiriam. Esta aula dialoga diretamente com o autoestudo de Arquitetura Cliente-Servidor de Programação 2 (previsto para a mesma semana) e antecipa a discussão de padrões MVC/MVP que será formalizada na aula seguinte daquela disciplina (14/09), reforçando de forma interdisciplinar a relação entre arquitetura de software e arquitetura de dados.
08/09/2026	Prova P1	Aplicação da prova P1
15/09/2026	Arquitetura de Banco de Dados - Gerenciamento de transações e controle de concorrências	Explicação teórica sobre mecanismos de controle de concorrência e funcionamento do ambiente transacional. Atividade prática relacionada a Dead locks e bancos de dados e implementação de semáforos.

Apresentação da Disciplina

22/09/2026	Arquitetura de Banco de Dados - Mecanismos de Proteção e Recuperação à Falhas	Estudos das técnicas de proteção aos dados e dos mecanismos de acesso do BD. Estudo das técnicas de recuperação à falhas e logs do banco. Prática para configurar estes parâmetros diretamente no BD.
29/09/2026	Bancos de Dados Distribuídos	Componentes de Um BD distribuído e sua principal diferença para os BDs tradicionais.
06/10/2026	Bancos de Dados Distribuídos - Replicação	Estudo dos diferentes mecanismos de replicação para os BD relacionais.
2 - Autoestudo	SQL Avançado - Permissões em Bancos de Dados.	Atividade prática com estruturas de controle de operações no BD. Criar usuários no BD, modificar e conceder autorizações. Exercícios práticos. Aula remota devido à EFAPI
20/10/2026	Banco de Dados Distribuídos - Recuperação à Falhas	Explicação sobre o funcionamento e as técnicas para recuperação à falhas envolvidas em BDs distribuídos.



Apresentação da Disciplina



27/10/2026	Bancos de Dados Não Relacionais	Classificação e principais características de BDs não relacionais. Introdução aos BDs NoSQL e seus diferentes modelos de dados. Estudo Prático utilizando BDs chave-valor.
03/11/2026	Bancos de Dados Não Relacionais	Estudo de BDs NoSQL orientados à documentos. Prática envolvendo o MongoDB.
10/11/2026	Bancos de Dados Não Relacionais	Estudo de BDs NoSQL orientados à documentos à grafos. Prática envolvendo Neo4j.
17/11/2026	Prova P2	Aplicação da Prova P2
24/11/2026	Apresentação do Trabalho Final	Apresentação do Trabalho Final
01/12/2026	Seminário e apresentação dos TCCs	Seminário e apresentação dos TCCs



Apresentação da Disciplina

- Avaliação



Apresentação da Disciplina

- Avaliação
 - Assiduidade em sala de aula de no mínimo 75%;
 - Avaliação escrita/prática (P1 e P2 com peso 10);
 - Participação em aula e exercícios (PE peso 10);

$$A1 = (P1 + P2) * 0,3 + T1 * 0,4$$

- Nota **A1** $\geq 7,0$ APROVADO
- Nota **A1** ≥ 4 e Nota **A1** < 7 realizada A2
- As duas provas poderão ter trabalhos de recuperação, de acordo com o desempenho da turma e avaliação do professor.
- Além disso, ao longo do semestre poderão ser propostos desafios e seminários que, conforme o desempenho do estudante, poderão somar até 1,0 (um) ponto adicional na média final da disciplina (A1).



Apresentação da Disciplina

- Avaliação A2



Apresentação da Disciplina



- Avaliação A2
 - Caso necessário será realizada uma prova como exame, a prova irá abranger todo o conteúdo sem consulta.
 - A forma de correção da A2 tem um caráter mais crítico do que as outras provas por se tratar de um exame, ou seja, erros básicos não serão considerados. Lembrando que o aluno que ficar em A2 vai necessitar no mínimo 4,0 independente do resultado obtido até então.



Revisão SQL



- 1970 foi publicado “A Relation Model of Data for Large Shared Banks” que define o modelo relacional;
- 1974 é publicado SEQUEL (Structured English Query Language)
- 1986 SQL vira ISO
 - SQL-2 (92)
 - SQL-3 (99)



- Possui base pautada na álgebra relacional e no cálculo relacional de tupla
- Principais funcionalidades:
 - DDL – Definição de Dados
 - DML – Manipulação de Dados
 - Restrições de Integridade
 - Transações
 - Store Procedures





- Criação de um Banco de Dados
- SQL não define essa operação
 - Alguns Bds (maioria dos comerciais) possuem um comando de criação:
 - **CREATE DATABASE** nome_bd
- Remover um BD
 - **DROP DATABASE** nome_db



- Comandos para definição do esquema
- **CREATE TABLE**
 - Cria uma nova tabela, com suas definições estruturais e de integridade
- **ALTER TABLE**
 - Modifica as definições da tabela
 - Atributos chaves não podem ser removidos
 - Atributos NOT NULL não podem ser inseridos
- **DROP TABLE**
 - Remove uma tabela



- CREATE TABLE

- **CREATE TABLE** tabela (
 atributo_1 tipo_1,
 [atributo_n tipo_n],
 [**PRIMARY KEY** (atributo_, atributo_n)],
 [**FOREIGN KEY** (nome_atributo) **REFERENCES**
nome_tabela]
);

- Principais tipos

- Int, smallint, numeric (definição e tamanho), char, varchar (definição de tamanho), date, datetime
- Data → 'YYYY-MM-DD'



DDL

Name	Aliases	Description
<code>bigint</code>	<code>int8</code>	signed eight-byte integer
<code>bigserial</code>	<code>serial8</code>	autoincrementing eight-byte integer
<code>bit [(n)]</code>		fixed-length bit string
<code>bit varying [(n)]</code>	<code>varbit</code>	variable-length bit string
<code>boolean</code>	<code>bool</code>	logical Boolean (true/false)
<code>character [(n)]</code>	<code>char [(n)]</code>	fixed-length character string
<code>character varying [(n)]</code>	<code>varchar [(n)]</code>	variable-length character string
<code>date</code>		calendar date (year, month, day)
<code>double precision</code>	<code>float8</code>	double precision floating-point number (8 bytes)
<code>integer</code>	<code>int, int4</code>	signed four-byte integer
<code>numeric [(p, s)]</code>	<code>decimal [(p, s)]</code>	exact numeric of selectable precision
<code>smallint</code>	<code>int2</code>	signed two-byte integer
<code>text</code>		variable-length character string
<code>time [(p)] with time zone</code>	<code>timetz</code>	time of day, including time zone
<code>timestamp [(p)] with time zone</code>	<code>timestampz</code>	date and time, including time zone



- CREATE TABLE
 - **CREATE TABLE** alunos (
matricula NUMERIC(10),
nome VARCHAR(120),
sexo SMALLINT,
PRIMARY KEY (matricula)
);

- Restrições (constraints) são construções previstas no SQL utilizados para definir um nível de controle mais apurado sobre os dados e seu domínio
- Geralmente, restrições podem ser definidas de duas maneiras
 - Através de DDL na construção de tabelas
 - Unique, not null, check
 - Adicionados posteriormente sob demanda
 - constraint



- Checagem (check)
 - Permitem a especificação de valores válidos ou não para o campo
 - Podem referenciar outros campos
 - Avaliam uma expressão booleana

- Exemplo

```
create table Empregados (  
  
    codEmp integer,  
    nome varchar(40) not null,  
    CPF numeric(11) not null unique,  
    idade integer check (idade between 16 and 90),  
    estadoCivil char(10) check (estadoCivil in  
        ('solteiro', 'casado', 'viúvo', 'desquitado',  
        'divorciado')),  
    salario numeric(8,2) check (salario > 0),  
    tempoServiço integer,  
    codGer integer,  
    codDepto integer,  
    ...
```

- Exemplo

```
...  
constraint EmpPk primary key (codEmp),  
constraint EmpCodg foreign key (codGer)  
    references Empregados on update cascade  
                        on delete set null,  
constraint EmpDept foreign key (codDepto)  
    references Departamentos on update cascade  
                        on delete no action,  
constraint IdadeTS check(tempoServico < idade),  
constraint EmpGer check(codEmp < > codGer),  
constraint SalarioGerente  
    check(salario < (select salario  
                    from Empregados e  
                    where e.codEmp = codGer))  
);
```

- Exemplo

```
...  
constraint EmpPk primary key (codEmp),  
constraint EmpCodg foreign key (codGer)  
    references Empregados on update cascade  
                        on delete set null,  
constraint EmpDept foreign key (codDepto)  
    references Departamentos on update cascade  
                        on delete no action,  
constraint IdadeTS check(tempoServico < idade),  
constraint EmpGer check(codEmp < > codGer),  
constraint SalarioGerente  
    check(salario < (select salario  
                    from Empregados e  
                    where e.codEmp = codGer))  
  
);
```

- Exemplo

```
create table Departamentos (  
  
    codDepto integer,  
    nome varchar(20) check (nome in ('Vendas',  
        'Pessoal', 'Finanças', 'Administrativo')),  
    Andar integer check (andar between 1 and 10),  
    orçamento numeric(20,2),  
  
    constraint DeptPk primary key (codDepto),  
    constraint ControleOrç  
        check ( orçamento > =  
            (select sum(salário)  
              from Empregados e  
              where e.codDepto = codDepto))  
);
```



- Exemplo

```
create table Departamentos (  
  
    codDepto integer,  
    nome varchar(20) check (nome in ('Vendas',  
        'Pessoal', 'Finanças', 'Administrativo')),  
    Andar integer check (andar between 1 and 10),  
    orçamento numeric(20,2),  
  
    constraint DeptPk primary key (codDepto),  
    constraint ControleOrç  
        check( orçamento > =  
            (select sum(salário)  
                from Empregados e  
                where e.codDepto = codDepto))  
  
);
```



- Exemplo (Inclusão posterior)

```
ALTER TABLE departamentos ADD CONSTRAINT  
nome_departamento CHECK (nome IN ('Vendas',  
'Pessoal', 'Financas'))
```

```
ALTER TABLE fun_dep ADD CONSTRAINT  
unicidade_departamento UNIQUE  
(id_departamento, id_funcionario)
```



- DROP TABLE
 - **DROP TABLE** nome_tabela;
 - Exemplo
 - **DROP TABLE** alunos;

- ALTER TABLE

- **ALTER TABLE** nome_tabela **ADD COLUMN** nome_atributo tipo [, nome_atributo_n tipo_n]
- **ALTER TABLE** nome_tabela **DROP COLUMN** nome_atributo
- **ALTER TABLE** nome_tabela **MODIFY COLUMN** nome_atributo tipo [, nome_atributo_n tipo_n]
- **ALTER TABLE** nome_tabela [**ADD | DROP**] **CONSTRAINT** nome_ref tipo_ref;
- [**ADD|DROP**] [**PRIMARY KEY...**| **FOREIGN KEY ...**]





- ALTER TABLE
 - Exemplo
 - **ALTER TABLE** ambulatorios **ADD** nome varchar(30)
 - **ALTER TABLE** medicos **DROP PRIMARY KEY**
 - **ALTER TABLE** pacientes **DROP COLUMN** doenca,
DROP COLUMN cidade
 - **ALTER TABLE** funcionarios **ADD FOREIGN KEY**
(nroa) **REFERENCES** ambulatorios



- Utilizada para a manipulação dos dados
- Comandos
 - **INSERT**
 - **UPDATE**
 - **SELECT**
 - **DELETE**
- São instruções declarativas
 - Manipulação de conjuntos
 - Baseia-se no que deve ser feito





- Inserção dos dados
 - **INSERT INTO** nome_tabela [(lista_atributos)] **VALUES** (lista_valores | (**SELECT** ...))
 - Exemplo
 - **INSERT INTO** ambulatorios **VALUES** (1, 1, 30)
 - **INSERT INTO** medicos (codm, nome, idade, especialidade, CPF, cidade) **VALUES** (4, 'Carlos', 28, 'ortopedia', 110000110000, 'Joinville');



- Alteração de dados
 - **UPDATE** nome_tabela **SET** nome_atributo_1 = valor_1 [, nome_atributo_n = valor_n]
[**WHERE** condicao]
 - Condição
 - Atributo (>, <, >=, <=, <>, =) [atributo | calculo | valor];
 - Condição1 **AND** Condicao2 **OR** Condicao3



- Alteração de dados
 - **UPDATE** nome_tabela **SET** nome_atributo_1 = valor_1
[, nome_atributo_n = valor_n]
[**WHERE** condicao]
 - Exemplo
 - **UPDATE** medicos **SET** cidade = 'Florianópolis'
 - **UPDATE** ambulatorios **SET** capacidade =
capacidade+5, andar = 3 **WHERE** nroa = 2



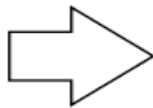
- Exclusão de dados
 - **DELETE FROM** nome_tabela
[**WHERE** condicao]
 - Exemplo
 - **DELETE** medicos **WHERE** cidade =
'Florianópolis'

DML – Select

- Consulta de dados
 - **SELECT** lista_atributos **FROM** tabelas
[**WHERE** condição]

- Mapeamento para Álgebra Relacional

select $a_1, \dots, a_n$
from t
where c



$\pi_{a_1, \dots, a_n} (\sigma_c (t))$

- Ex:
 - **SELECT** * **FROM** Medicos

Exercícios



Clientes	<u>CPF</u>	nome	origem	fone
-----------------	------------	------	--------	------

Hóspedes	<u>CPF</u>	motivo	placaVeículo	nroAcomp	dataEnt	dataSai	<i>quarto</i>
-----------------	------------	--------	--------------	----------	---------	---------	---------------

Quartos	<u>número</u>	frigobar	<i>tipo</i>
----------------	---------------	----------	-------------

TiposQuartos	<u>código</u>	descrição	preço
---------------------	---------------	-----------	-------

Reservas	<i>cliente</i>	<u>quarto</u>	<u>dataEnt</u>	dataSai
-----------------	----------------	---------------	----------------	---------

Cardápios	<u>código</u>	descrição	preço
------------------	---------------	-----------	-------

Consumo	<u>ID</u>	<i>hóspede</i>	<i>itemCardápio</i>	data	qtde
----------------	-----------	----------------	---------------------	------	------

Criar o BD conforme as regras disponíveis no portal;
Popular o BD;



Exercícios

- 1) Buscar o nome, CPF e origem de todos os clientes.
- 2) Buscar nome, CPF e o motivo de hospedagem de todos os clientes.
- 3) Buscar a descrição e o preço dos itens do cardápio que custam mais de 30 reais.
- 4) Buscar para todos os quartos as seguintes informações: o numero do quarto, a descrição, o preço e se tem frigobar.
- 5) Buscar o motivo e o número de acompanhantes de todos os hóspedes que vieram de carro.
- 6) Buscar o nome, CPF e telefone dos clientes de '*Florianópolis*', '*Joinville*' e '*Blumenau*'.
- 7) Buscar o motivo da hospedagem, o número de hóspedes por motivo, a média de acompanhantes e a média de dias que os hóspedes ficaram no hotel.
- 8) Buscar o número de quartos que não possuem reserva na data de 29/07/2019.